Learning Where to Sharpen: Selective Boundary Refinement for Diffusion-Based Monocular Depth

TODO Given Name Surname
TODO dept. name of organization
TODO name of organization
TODO City, Country
TODO email

***Abstract*—Diffusion-based monocular depth estimators are accurate on smooth surfaces but blur the depth discontinuities at object boundaries. A natural remedy is to sharpen the predicted depth at the contours found by a segmentation model, yet applying it at every contour lowers boundary F1. Observing that this loss arises from sharpening contours that carry no depth step, we propose to keep the sharpening operation fixed and to learn only which contours it should be applied to. In experiments, the method improves boundary F1 on NYUv2 by 11.9 percent, and the same selector, applied without retraining to a depth model of a different architecture and output representation, improves it by 27.8 percent.**

***Index Terms*—monocular depth estimation, depth boundaries, segmentation, diffusion models**

I. Introduction

Recent work repurposes the prior knowledge of large image generation models for monocular depth estimation. Marigold [1] fine-tunes the image diffusion model Stable Diffusion on synthetic RGB-D data and obtains a depth estimator that generalises well from a small amount of training data. Lotus [2] develops this further, replacing the multi-step diffusion process with a single-step prediction and accelerating inference substantially while keeping its accuracy. Marigold V2 [3] raises accuracy further by building on a diffusion transformer for image editing.

These methods, however, blur the depth discontinuity at object boundaries. This is thought to stem from the smoothness prior of the generative model and from predicting in a latent space of reduced resolution, and it spreads the step at an occlusion boundary over several pixels. Pixel-averaged metrics such as AbsRel hardly register the degradation, since boundary pixels are a small fraction of the image; it becomes apparent under boundary-specific metrics [5].

Improving boundaries by post-processing is attractive because it leaves the depth model untouched and applies to the output of any model. Prior work has learned how to correct the depth, through a displacement field that re-samples pixels around occlusion boundaries [7] or through edge-guided post-processing [9].

A simpler remedy draws on image segmentation. A segmentation model such as Segment Anything [11] partitions an image into objects and regions and extracts their contours with high accuracy, and the contour of an object often coincides with a depth discontinuity. The predicted depth can then be refilled from both sides of a contour, turning a blurred step into a sharp one; we call this sharpening. Applied at every contour on Marigold V2, however, it lowers boundary BF_1 by 1.9%. Most segmentation contours (79.8%), such as wallpaper patterns and floor seams, are contours that carry no depth step, and sharpening there introduces a discontinuity the scene does not have.

This suggests that whether sharpening helps is decided less by where the contours lie than by which of them it is applied to. Indeed, selecting with ground truth only the contours that lie on true depth steps turns the effect into a gain without moving any contour. We therefore keep the sharpening operation fixed and learn only which contours to apply it to. The selector is a small U-Net that takes the RGB image, the predicted depth and the contours as input, while the depth model and the segmentation model stay frozen. Since the selector is the only trained component, the improvement is attributable to learning the selection. On NYUv2 the method improves boundary BF_1 by 11.9%, and the same selector, applied to Lotus without retraining, improves it by 27.8%.

Our contributions are threefold:

1) A post-processing method that fixes the sharpening operation and learns only which contours to apply it to (Sec. III).
2) Evidence that it improves boundary accuracy beyond training-free controls and classical edge-preserving filters (Sec. IV).
3) Evidence that it transfers across depth models without retraining but not across datasets (Sec. IV).

II. Related Work

A. Diffusion-based monocular depth estimation

Marigold [1] fine-tunes the image diffusion model Stable Diffusion on synthetic RGB-D data to estimate affine-invariant depth. Lotus [2] predicts the depth itself rather than noise and replaces the multi-step process with a single step at a fixed timestep, improving both accuracy and speed. Garcia et al. [4] showed that correcting Marigold's inference procedure and combining single-step inference with end-to-end fine-tuning makes it orders of magnitude faster without loss of accuracy. Marigold V2 [3] builds on a diffusion transformer for image editing and estimates affine-invariant log depth in a single step.

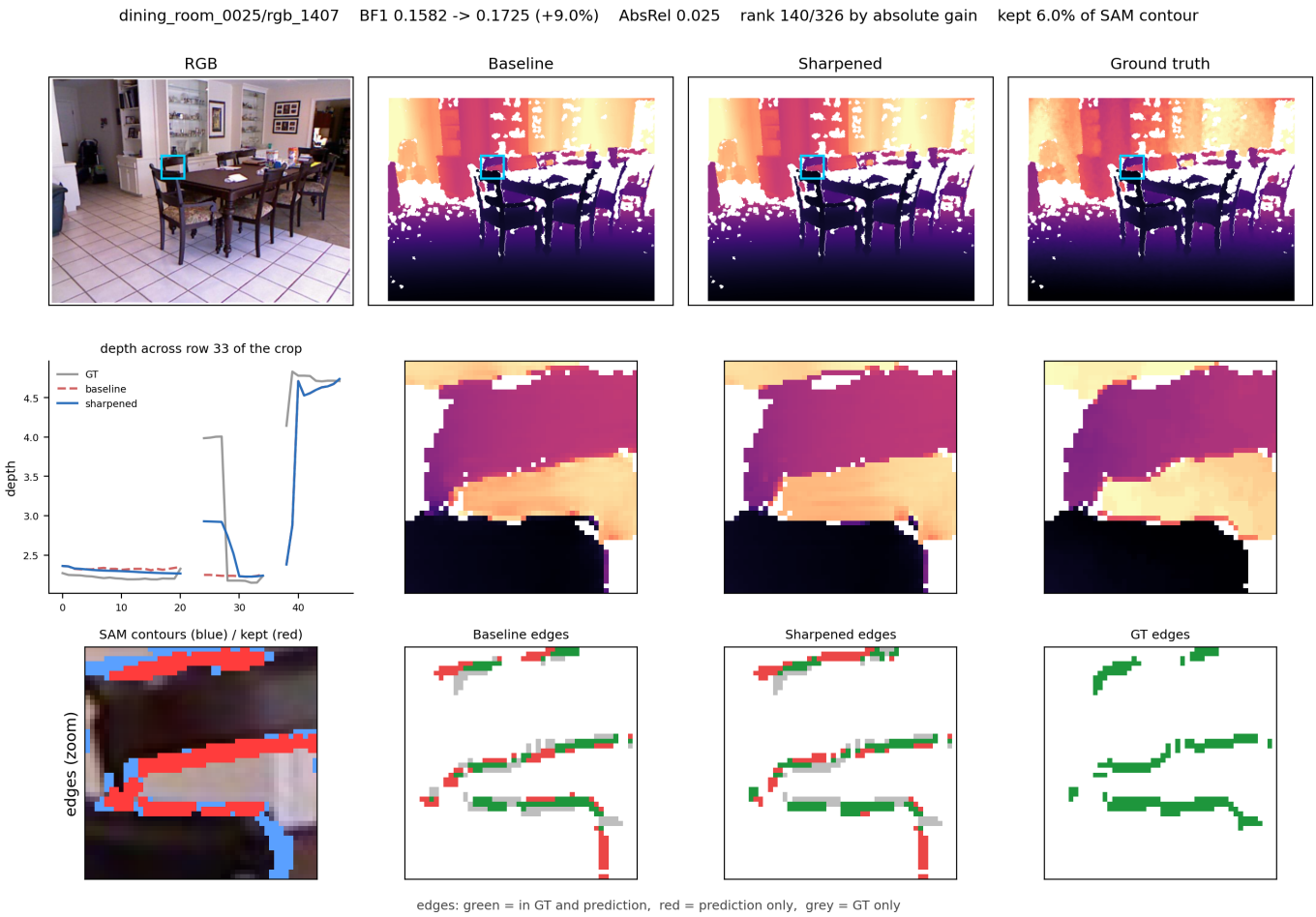


Fig. 1. What the method does, on a dining-room frame. Top: the whole frame; nothing outside the bands around the selected contours changes. Middle: a 48 px crop at the chair-back boundary, with a depth profile across it. The baseline misses both steps ground truth has; the sharpened map reproduces them. The three maps in the crop share a colour scale taken from it. Bottom: the depth discontinuities each map contains, which is what BF_1 compares; green is in both, red is in the prediction only, grey is in ground truth only. The frame is ranked 140th of the 326 whose AbsRel is at or below the split median of 0.0303, by absolute BF_1 gain; its relative gain of $+9.01\%$ is below the split median of $+13.15\%$.

All of these improve the depth model itself; this work leaves the model untouched and post-processes its output.

B. Improving depth boundaries

Ramamonjisoa et al. [7] learn a 2D displacement field that re-samples pixels around occlusion boundaries, improving the output of any depth estimator by post-processing. It uses NYUv2 with the same 795 training and 654 test frames and is the closest prior work. SharpNet [8] builds occluding-contour estimation into the training of the depth network, and Peng et al. [9] reduce the degradation near occlusion boundaries in self-supervised monocular depth with an edge-guided post-processing step. Muthukkumar [10] fuses a monocular estimate with depth from a neural radiance field to recover detail. All of these design or learn how to correct the depth; this work fixes the correction and learns only where to apply it. Since no trained weights are released for [7], we do not compare against it directly.

C. Segmentation and depth estimation

Segment Anything [11] is a segmentation model that extracts arbitrary regions of an image without being told the object class. Its contours capture object boundaries well, but many of them, such as wallpaper patterns and floor seams, carry no depth discontinuity. This work uses these contours as candidates for depth boundaries and learns to select those that carry a depth step. To our knowledge, no prior work uses the contours of a segmentation model as candidates for depth boundaries.

D. Boundary metrics

Pixel-averaged metrics such as AbsRel and δ_1 hardly reflect blurred boundaries, since boundary pixels are a small fraction of the image. Depth Pro [5] proposed a scale-invariant boundary metric, BF_1, that declares a depth step where the ratio of neighbouring depths exceeds a threshold, sweeps that threshold from 5% to 25%, and scores the agreement with F1. SMD-Nets [6] proposed the Soft Edge Error (SEE)

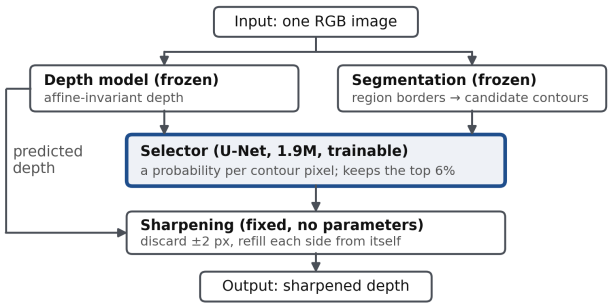


Fig. 2. Pipeline of the method. From the input RGB image, the frozen depth model predicts depth and the frozen segmentation model extracts candidate contours. The selector maps the image, the depth and the contours to a probability for each contour pixel and keeps only a fixed fraction of the most probable. Sharpening is applied at the kept contours alone, leaving every other pixel unchanged. Only the selector is trained.

for evaluating stereo, measuring the error at ground-truth boundary pixels while tolerating small displacements, and Marigold V2 [3] also uses it. We use both and propose no new metric.

III. Method

A. Overview

Fig. 2 gives an overview of the method. The input is a single RGB image. The depth model predicts affine-invariant depth, and the segmentation model extracts candidate contours from region borders. The selector takes the image, the predicted depth and the contours, outputs a probability for each contour pixel, and keeps a fixed fraction of the most probable. Finally, the barrier sharpening of Sec. III-B is applied at the kept contours. Only the selector is trained: the depth model and the segmentation model are frozen, and barrier sharpening has no learned parameters. Contours are selected because many of them, such as wallpaper patterns and floor seams, carry no depth step, and sharpening there creates a discontinuity the scene does not have (Sec. IV-B).

B. Barrier sharpening

Diffusion-based depth estimators mix foreground and background depth at object boundaries, turning a step that should change within one pixel into a slope over several. If the boundary is known, the step can be restored by discarding the mixed pixels around it and refilling each side from its own depth alone. We take the boundary from the contours of a segmentation model and, because each contour acts as a barrier to interpolation, call this operation *barrier sharpening*.

Let z be a predicted depth map, V its valid pixels and $C \subseteq V$ a set of contour pixels. The operation discards the band

$$B = \{\, p \in V \mid \operatorname{dist}(p, C) \le r_b \,\}, \tag{1}$$

labels the connected components of $V \setminus B$, so that the two sides of a contour receive different labels, and floods those labels inward one pixel at a time until B is covered, giving every $p \in B$ a label $\ell(p)$. Each discarded pixel is then refilled from its own side only,

$$\hat{z}(p) = \frac{1}{|N(p)|} \sum_{q \in N(p)} z(q), \tag{2}$$

where $N(p) = \{\, q \notin B \mid \|q - p\| \le r_f,\ \ell(q) = \ell(p) \,\}$. Pixels outside B are unchanged. The component labels come from the contours themselves, not from the segmentation masks. The operation has only two settings, the band width r_b and the refill radius r_f, and learns nothing. We require $r_f > r_b$, or $N(p)$ is empty and the operation leaves the map unchanged.

The separation of the two sides holds only where a contour closes or reaches the image border. Where it ends inside the image the valid region stays connected around that end, so both sides of the band belong to one component there and one-sided interpolation does not hold. Selection makes this more common, since keeping only part of the contour pixels leaves fragments rather than whole outlines. Both effects weaken the repair, and both are present in every number we report.

C. The selector

Inputs and output. The selector is a U-Net that takes the RGB image (3 channels), the predicted depth (1) and the contour mask (1), and emits one logit per pixel. The prediction is standardised per image. For a model that emits disparity v rather than log depth we apply $-\log\max(v, 10^{-3})$ first: disparity rises as depth falls, so feeding it raw would invert the signal, and an affine-invariant disparity can be non-positive, which the clamp handles. This is monotone in depth but not an affine map of log depth: the unknown shift in v survives the logarithm as a nonlinearity that per-image standardisation cannot remove. We apply the selector unchanged regardless, so the Lotus results in Table II include the effect of this approximation.

Retention. Rather than thresholding the probability, we keep a fixed fraction ρ of the contour pixels per image, in order of probability. A fixed threshold keeps a different amount depending on how many contours a scene happens to contain, so the budget the repair spends would drift from frame to frame. ρ is set to the share of positive contour pixels in the training data.

Labels and loss. A contour pixel is positive if it lies within 1 px of a depth discontinuity in ground truth, where a discontinuity is a step whose ratio of neighbouring depths exceeds a threshold τ. The selector is trained with binary cross-entropy averaged over the contour pixels only, since pixels off the contours are never candidates. Ground truth is used only to generate these labels and never appears in the inference path.

IV. Experiments

A. Setup

We evaluate on the 654-frame NYUv2 [12] test split, scene-disjoint from the training frames. The two transfer sets of Sec. IV-E are the 800-frame ScanNet [13] validation sample

used by [3] and a 300-frame scene-level hold-out of Hypersim [14]. All three store depth as 16-bit millimetres and are read with each dataset's own validity mask and depth range; NYUv2 additionally uses the Eigen crop. The metric is the scale-invariant boundary BF_1 of Depth Pro [5]. We report the relative change in BF_1 against each base model's own unsharpened prediction. Predictions are aligned to ground truth in their own space before scoring (log depth for Marigold V2, disparity for Lotus) and the alignment affects every variant equally.

Depth models. We evaluate two models that are as far apart as the family allows: Marigold V2, a 21.3B-parameter diffusion transformer fine-tuned with 4-bit QLoRA that emits affine-invariant log depth, and Lotus, an 868M-parameter UNet that emits disparity. Neither reports BF_1 on NYUv2, so we measure the baselines ourselves.

Choice of benchmarks. BF_1 compares neighbouring pixels, so it needs ground truth dense enough for both to be valid, and three of the five benchmarks these models are evaluated on do not provide it: KITTI [15] leaves 13.2% of adjacent pairs valid and ETH3D [16] 4.1%, while DIODE [17] is dense but its evaluation protocol masks the depth gradients this method operates on, removing 99.4% of indoor discontinuities. We therefore evaluate on NYUv2 and ScanNet, and additionally on Hypersim as a third transfer set.

Settings of the method. Barrier sharpening uses $r_b = 2$ and $r_f = 3$. The selector is a four-level U-Net with 1.9M parameters. On the training frames 6.066% of contour pixels lie within 1 px of a true step, so the retention is $\rho = 6\%$. The discontinuity threshold is $\tau = 10\%$, chosen on a validation split rather than on the test set: scoring BF_1 on the 141 frames of the 40 scenes held out of the 795 training frames, 10% gives $+12.74\%$ against $+8.66$, $+6.29$ and $+5.28\%$ for 15, 20 and 25%. Raising it thins the positives (from 6.07% of contour pixels to 2.13%) so the fixed budget is then spent mostly on negatives. Every hyperparameter of the method was set on the validation split; the one exception is a control, and Sec. IV-E states it there. The 795 NYUv2 training frames are split by scene into 654 for training and 141 for validation; the deployed checkpoint is the best-validation epoch and has seen only those 654, which are a different set from the 654 test frames. Training takes 16 minutes and 3.5 GB of VRAM on one RTX 4090.

Implementation. The selector is trained with AdamW at a learning rate of 3×10^{-4}, weight decay 10^{-4} and a cosine schedule, for 40 epochs at batch size 4. Frames are used at their native 480×640 with no resizing and no augmentation: the band and the refill radius are defined in pixels, so rescaling would change what $r_b = 2$ means. Contours come from Segment Anything [11] with the ViT-H checkpoint in automatic mode, a 48×48 prompt grid and its default quality filter; "grid 16" and "grid 32" in Sec. IV-E vary that grid. A frame's contour set is the union of the one-pixel boundaries of the returned masks, not the union of the masks, which would fill the frame. Timings are on one RTX 4090.

TABLE I

WHY POST-HOC SHARPENING FAILS ON MARIGOLD V2, AND WHAT FIXES IT. NYUV2, 654 FRAMES. ORACLE SELECTION USES GROUND TRUTH TO CHOOSE A SUBSET OF THE SEGMENTATION CONTOURS AND MOVES NONE OF THEM.

Contours sharpened	share kept	BF_1 change
all segmentation contours	100%	-1.90%
oracle selection, within 1 px of a step	6.07%	$+\mathbf{43.39}\%$

TABLE II

MAIN RESULT. FOUR SEEDS; $\pm$ IS ONE STANDARD DEVIATION. THE SAME FOUR SELECTORS ARE APPLIED TO BOTH MODELS AND WERE TRAINED ONLY ON THE FIRST. IN ABSOLUTE TERMS THE SELECTOR TAKES BF_1 FROM 0.0884 TO 0.0989 AND FROM 0.0693 TO 0.0886.

Base depth model	BF_1	no sel.	+ selector
Marigold V2, 21.3B, log	0.0884	-1.90%	$+\mathbf{11.92} \pm \mathbf{0.90}\%$
Lotus, 868M, disparity	0.0693	$+10.31\%$	$+\mathbf{27.83} \pm \mathbf{1.27}\%$

B. Why selection is needed

Applied at every contour that Segment Anything draws on NYUv2, barrier sharpening lowers Marigold V2's boundary BF_1 by 1.90% (Table I). This is specific to a model whose boundaries are already reasonable: on Lotus, whose boundaries start blunter (baseline BF_1 0.0693 against 0.0884), the same unselected sharpening *gains* 10.31% (Table IV).

Measuring from each true depth discontinuity to the nearest contour gives a median of 1.00 px, with 66.9% within 1 px and 76.9% within 2 px. The contours are where the boundaries are, but 79.8% of the contour pixels have no depth step anywhere within an eight-pixel window; these are wallpaper patterns, floor seams and poster edges. Sharpening there introduces a discontinuity into a region that has none.

The 79.8% uses a generous eight-pixel window; the selection below uses a strict one-pixel tolerance. Of the 22,167 segmentation contour pixels in an average frame (10.6% of its valid pixels) 1,345, or 6.07%, lie within 1 px of a true discontinuity. An oracle that keeps exactly those, and leaves every kept contour where it is, moves the effect from -1.90% to $+43.39\%$ (Table I). The loss is therefore attributable to which contours are sharpened rather than to where they lie, and the selector of Sec. III-C learns that selection.

C. Main result

Table II gives the result on two depth models that share no architecture, no parameter scale and no output representation. Four selectors are trained, differing only in seed, all on Marigold V2's predictions. The same four are applied to Lotus with no retraining, adding only the disparity-to-log-depth conversion; the deviation reported there is the spread of those four selectors on Lotus, not of four Lotus-trained models.

All four seeds clear $+10\%$ on Marigold V2, with a 95% confidence interval whose lower bound is $+10.49\%$. Pixel-wise accuracy does not regress either. We measure it with AbsRel, the mean absolute error relative to the ground-truth

TABLE III

SOFT EDGE ERROR, FOR ONE OF THE FOUR SEEDS OF TABLE II (THE ONE SCORING $+12.49\%$, SECOND BEST OF THE FOUR) APPLIED UNCHANGED TO BOTH MODELS. LOWER IS BETTER; PERCENTAGES ARE RELATIVE TO EACH MODEL'S OWN BASELINE. "GT CONTOURS" SHARPENS AT THE GROUND-TRUTH DISCONTINUITIES THEMSELVES AND IS NOT REACHABLE BY SELECTING AMONG SEGMENTATION CONTOURS; THE ORACLE SELECTION OF TABLE I IS A DIFFERENT AND MUCH SMALLER BOUND.

	BF_1	SEE_3	SEE_5	SEE_7
Marigold V2				
baseline	0.0884	0.1148	0.0882	0.0758
+ selector	$+12.49\%$	-3.34%	-2.89%	-2.39%
relocated	-0.50%	$+0.32\%$	$+0.46\%$	$+0.52\%$
GT contours	$+209.8\%$	-11.44%	-5.09%	-2.91%
Lotus				
baseline	0.0693	0.1403	0.1147	0.1014
+ selector	$+28.18\%$	-5.18%	-5.54%	-5.48%
relocated	-0.57%	$+0.20\%$	$+0.23\%$	$+0.24\%$
GT contours	$+279.0\%$	-12.96%	-9.81%	-8.73%

TABLE IV

TRAINING-FREE CONTROLS, MEASURED SEPARATELY ON EACH BASE MODEL. BF_1 CHANGE. THE RELOCATED ROW IS THE MEAN OF THREE INDEPENDENT RELOCATIONS; THE TWO RANDOM ROWS ARE MEANS OF THREE DRAWS, EACH SPENDING THE SELECTOR'S OWN PER-FRAME BUDGET. THE FILTER ROW IS THE BEST OF A GUIDED AND A JOINT BILATERAL FILTER OVER FOUR RADII, JOINT BILATERAL AT RADIUS 1 ON BOTH.

Method	Marigold V2	Lotus
none (all contours)	-1.90%	$+10.31\%$
relocated (same sparsity)	-0.50%	-0.57%
random pixels	$-24.96 \pm 0.53\%$	$-22.11 \pm 1.17\%$
random blocks	$-1.04 \pm 0.15\%$	$-0.03 \pm 0.36\%$
best hand-crafted threshold	$+1.25\%$	$+10.46\%$
joint bilateral filter, $r{=}1$	-4.75%	-7.15%
trained selector	$\mathbf{+11.92\%}$	$\mathbf{+27.83\%}$
learning contributes	**10.7** pt	**17.4** pt

depth (lower is better), and δ_1, the share of pixels whose prediction is within a factor of 1.25 of ground truth (higher is better). AbsRel moves from 0.03881 to 0.03860 and δ_1 from 0.97969 to 0.98000, both slightly in the better direction.

Per-frame scoring shows the mean is not carried by a few frames: 484 of 651 frames improve (74.3%), with a median relative gain of $+13.15\%$ and first and third quartiles -0.19% and $+31.57\%$; a quarter of frames are therefore slightly worse. Three frames with a baseline BF_1 of zero are excluded, as their relative gain is undefined.

D. Evaluation with Soft Edge Error

An improvement visible in one metric may be an improvement in that metric's definition. We therefore also report Soft Edge Error [6], the boundary metric Marigold V2 adopts, transcribed from its implementation. SEE_k averages, over ground-truth boundary pixels, the smallest absolute error between the prediction and any ground-truth depth in a $k \times k$ neighbourhood; it forgives a displaced boundary and punishes a blurred one. Unlike BF_1 it is an absolute error in depth units, so it is not scale-invariant and the prediction must be aligned first. Lower is better.

The two metrics agree on every variant of both models: the selector improves both, the relocated control worsens both, and sharpening at GT contours is best on both. The improvement is therefore not an artefact of BF_1's definition.

Their magnitudes are not comparable and should not be read as such. SEE is an absolute error averaged over boundary pixels, floored by how accurate the model's depth values are there, so sharpening at GT contours raises BF_1 by 209.8% while lowering SEE_5 by 5.09%.

E. Analysis

This subsection analyses where the improvement comes from and how far it holds. We first compare against training-free controls to confirm that the gain comes from the learned selection. We then measure the contribution of the segmentation model and test how the selector generalises to other datasets. Finally, we report the cost.

Controls. The gain could come from the sharpening rather than from the learning, or from simply being sparse. Table IV measures six training-free alternatives, each on the base model's own predictions.

The edge-preserving filter row is the classical alternative: a guided or joint bilateral filter with the image as guide. Both lose on both models, monotonically in the radius, from -4.75% and -7.15% at the gentlest setting to -50.14% and -45.30%. A filter can only smooth, and a smoothed boundary is what BF_1 counts against; discarding the band and refilling each side from itself is what restores a step instead.

The hand-crafted control thresholds a local descriptor of the predicted depth's log-gradient. We select its feature by the final metric on the test split rather than by ranking quality on held-out data: over four candidates the AUC ordering and the BF_1 ordering are *inverted*, and the feature with the worst AUC (0.801) gives the best BF_1 ($+1.25\%$) while the best AUC (0.876) gives the worst (-2.59%). That gives the control an advantage the method does not have, which makes the comparison conservative.

Both random rows spend exactly the budget the selector spends, taken from its output frame by frame. Random-pixel selection is close on the two models (-24.96% and -22.11%), which is what a control that measures the repair operation rather than the depth model should do. Random blocks, which keeps the contour contiguous, costs almost nothing on either and nothing at all on Lotus: scattering the selection is most of what makes the pixel control so destructive, and choosing badly on its own incurs almost no cost.

Contribution of the segmentation model. Removing the segmentation model entirely, so that the selector must find boundaries itself from RGB and depth and every valid pixel is a candidate, gives $+7.30\%$. Both halves of this ablation come from a single seed, so the figure to compare against is that seed's $+12.49\%$: the segmentation is worth 5.19 points,

TABLE V

GENERALISATION. THE SELECTOR WAS TRAINED ON NYUV2; THE DEPTH MODEL IS MARIGOLD V2 IN EVERY ROW. PRECISION IS MEASURED AT THE SAME 6% RETENTION.

Evaluation data	no selection	+ selector	precision
NYUv2 (in-domain)	-1.90%	$\mathbf{+11.92\%}$	50.2%
ScanNet	$+3.26\%$	$+2.81\%$	17.4%
Hypersim	$+2.28\%$	-4.64%	55.8%

42% of that run's gain. This contribution arises on the recall side rather than the precision side. On segmentation contours the true steps number 1,345 pixels per frame and the budget of 1,330 can cover 98.9% of them; counting over the whole valid image there are 2,941, and the same budget caps recall at 45.2%. The segmentation earns its contribution by *shrinking the target set*.

Cross-dataset generalisation. The selector transfers across depth models but not across datasets. Table V applies the NYUv2-trained selector to ScanNet and Hypersim, using Marigold V2 throughout.

On neither held-out dataset does the selector beat using every contour. What fails is the transfer, not the datasets. A selector trained on 10,000 Hypersim frames reaches $+5.05\%$ on Hypersim itself, against $+2.28\%$ for no selection, at 88.1% precision on a 22.8% base rate; the same checkpoint reaches -2.23% on NYUv2 and $+0.39\%$ on ScanNet. Hypersim is not saturated either (sharpening at its own GT contours gives $+39.49\%$) so the method works in its domain as it does in NYUv2's, and stops working only when the domain changes between training and use.

Three further remedies failed: dropping the RGB channels, sweeping the retention from 2% to 20% on ScanNet, and training on an equal mixture of 795 NYUv2 and 795 Hypersim frames. Data volume does not account for the gap either. Cutting NYUv2 to 393 frames costs 2.2 points of gain ($+11.92\%$ to $+9.72\%$, two seeds), against the 9.1 points that separate in-domain from ScanNet.

Precision does not order the rows. The NYUv2-trained selector is more precise on Hypersim than in its own domain (55.8% against 50.2%) yet scores worst there, and precision is not comparable across datasets in any case: the share of contour pixels on a real step is 6.07% on NYUv2 and 22.8% on Hypersim.

Cost. Table VI gives wall-clock time per frame on one RTX 4090, measured after warm-up on real frames. Marigold V2's transformer is loaded in 4-bit NF4, the released configuration's default, so its 21.3 billion parameters occupy roughly 10.6 GB and the model fits a 24 GB card; at bf16 it would not.

The selector costs 12.0 ms and the repair 18.3 ms, so what this work adds is 30.3 ms, 6% of the depth model's own 495 ms. The segmentation dominates everything else, at 534 ms to 2,603 ms depending on its point grid.

A coarse grid gives a comparable gain. Grid 16 reaches 92% of grid 48's gain for 20% of the segmentation cost, making the whole pipeline $3.0\times$ faster and cutting the time per

TABLE VI

PER-FRAME RUNNING TIME AND THE IMPROVEMENT OBTAINED. GRIDS 16 AND 48 ARE MEANS OVER THE FOUR SEEDS OF TABLE II WITH THEIR STANDARD DEVIATION; THE OTHER TWO ROWS ARE THE SINGLE SEED SCORING $+12.49\%$ THERE. THE SEGMENTATION POINT GRID IS THE ONLY SETTING WITH A SUBSTANTIAL EFFECT.

Configuration	ms/frame	BF_1 change	ms/BF_1 pt
depth model only	495	—	—
+ selector, no seg.	525	$+7.30\%$	**72**
+ selector, grid 16	**1,059**	$\mathbf{+11.02 \pm 0.64\%}$	**96**
+ selector, grid 32	1,996	$+11.17\%$	179
+ selector, grid 48	3,128	$+11.92 \pm 0.90\%$	262

point of BF_1 improvement to about one in 2.7. The 0.90-point gap equals the standard deviation across seeds, so a single pair of runs cannot settle the ordering; paired by seed, however, grid 48 is ahead in all four, by 0.90 ± 0.29 points (paired t-test, $p = 0.009$). The difference is consistent and small. Precision runs the other way across the grids (51.6%, 50.6%, 50.2% at grids 16, 32, 48, one seed), which says what the coarse grid is doing: it traces large objects and misses wallpaper patterns and floor seams, so a larger share of the candidates it produces are true boundaries. Grid 48 finds 3,200 more contour pixels per frame than grid 16 and yields 0.90 points of BF_1 with them, so most of those are false positives. This is the same mechanism as the ablation above (the segmentation helps by shrinking the target set) appearing along a different axis.

V. CONCLUSION

This paper proposed a post-processing method that improves the object boundaries blurred by diffusion-based monocular depth estimators without modifying the depth model. It fixes barrier sharpening, which restores depth steps at the contours of a segmentation model, and uses a small U-Net to select only the contours it is applied to.

The experiments yield three findings. First, sharpening at every contour lowers boundary accuracy, and the cause is not where the contours lie but that contours carrying no depth step are sharpened too. Second, learning which contours to sharpen improves the boundary BF_1 of Marigold V2 on NYUv2 by 11.9%, and the same selector applied to Lotus without retraining improves it by 27.8%; training-free controls and classical filters do not achieve this, and Soft Edge Error confirms it. Third, the selector transfers across depth models but not across datasets.

Limitations. The oracle selection of Sec. IV-B reaches $+43.39\%$ against our $+11.92\%$; the gap lies in the precision of the selection, 50.2% against 100%. For AbsRel and δ_1 we claim only that they do not regress. Supervision and BF_1 both refer to ground-truth depth steps; ground truth never enters inference, but the shared definition may bear on how the result is read. Making the selection independent of the training domain is the main open problem.

ACKNOWLEDGMENT

Claude Code (Anthropic) was used to implement the code and to write the initial draft of this paper. The author sub-

sequently reviewed the whole and takes responsibility for its content.

REFERENCES

[1] B. Ke, A. Obukhov, S. Huang, N. Metzger, R. C. Daudt, and K. Schindler, "Repurposing diffusion-based image generators for monocular depth estimation," in *CVPR*, 2024.

[2] J. He, H. Li, W. Yin et al., "Lotus: Diffusion-based visual foundation model for high-quality dense prediction," in *ICLR*, 2025.

[3] I. Pavlovic, T. Wandel, A. Obukhov, L. Bartolomei, A. Davydov, F. Tosi, M. Poggi, S. Süsstrunk, and D. Dai, "Marigold V2: Revisiting diffusion transformers for monocular depth estimation," arXiv:2609.08084, 2026.

[4] G. M. Garcia, K. Knaebel, C. Schmidt, D. de Geus, A. Hermans, and B. Leibe, "Fine-tuning image-conditional diffusion models is easier than you think," in *WACV*, 2025.

[5] A. Bochkovskii et al., "Depth Pro: Sharp monocular metric depth in less than a second," in *ICLR*, 2025.

[6] F. Tosi, Y. Liao, C. Schmitt, and A. Geiger, "SMD-Nets: Stereo mixture density networks," in *CVPR*, 2021.

[7] M. Ramamonjisoa, Y. Du, and V. Lepetit, "Predicting sharp and accurate occlusion boundaries in monocular depth estimation using displacement fields," in *CVPR*, 2020.

[8] M. Ramamonjisoa and V. Lepetit, "SharpNet: Fast and accurate recovery of occluding contours in monocular depth estimation," in *ICCV Workshops*, 2019.

[9] K.-S. Peng, G. Ditzler, and J. Rozenblit, "Edge-guided occlusion fading reduction for a light-weighted self-supervised monocular depth estimation," arXiv:1911.11705, 2019.

[10] A. Muthukkumar, "Bayesian monocular depth refinement via neural radiance fields," in *Proc. IEEE 8th Int. Conf. Algorithms, Computing and Artificial Intelligence (ACAI)*, 2025, pp. 488–492.

[11] A. Kirillov et al., "Segment Anything," in *ICCV*, 2023.

[12] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus, "Indoor segmentation and support inference from RGBD images," in *ECCV*, 2012.

[13] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner, "ScanNet: Richly-annotated 3D reconstructions of indoor scenes," in *CVPR*, 2017.

[14] M. Roberts et al., "Hypersim: A photorealistic synthetic dataset for holistic indoor scene understanding," in *ICCV*, 2021.

[15] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *CVPR*, 2012.

[16] T. Schöps, J. L. Schönberger, S. Galliani, T. Sattler, K. Schindler, M. Pollefeys, and A. Geiger, "A multi-view stereo benchmark with high-resolution images and multi-camera videos," in *CVPR*, 2017.

[17] I. Vasiljevic et al., "DIODE: A dense indoor and outdoor depth dataset," arXiv:1908.00463, 2019.